

Experiment Report

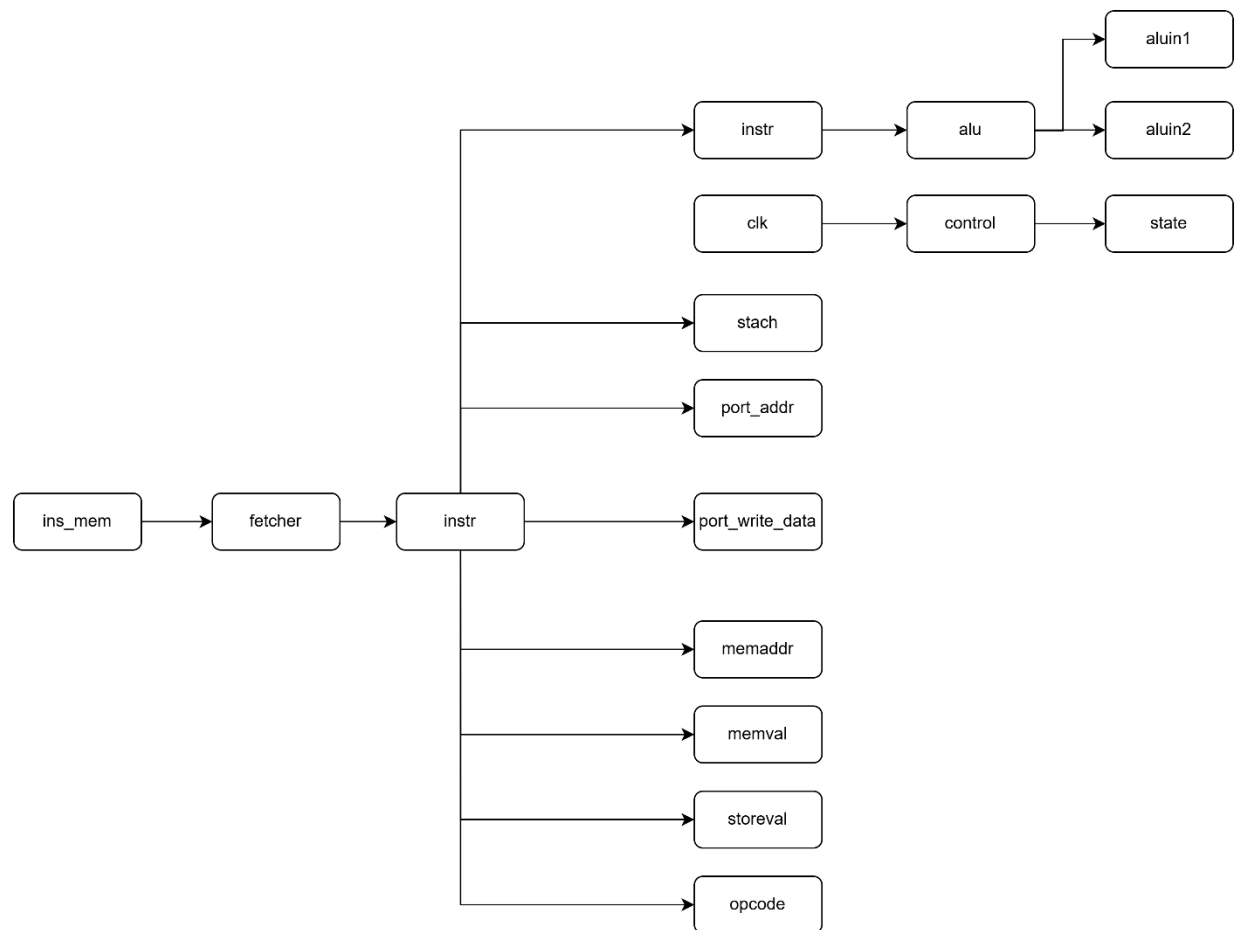
Student: Denisenko Sofiia 狄苏菲

Student id: 1820249051

Course: Computer Organization and Architecture

Experiment Topic: CPU

Idea and the diagram of the CPU



CPU data flow

The diagram shows the data flow starting from the “ins_mem” (instruction memory) and proceeds through various stages of processing until the instruction is completed.

Key Components and Their Roles:

- **ins_mem:** This represents the instruction memory, where program instructions are stored. It’s the starting point of the instruction processing pipeline.
- **fetcher:** The fetcher retrieves instructions from the “ins_mem”. The arrow indicates the flow of information from the memory to the fetcher.

- **instr:** This block represents the instruction register, where the currently fetched instruction is temporarily stored. The arrows leading from the fetcher show that the instruction output by the fetcher is now available in the instruction register. All other blocks get their input from the instruction register.
- **alu:** This is the Arithmetic Logic Unit. It performs arithmetic and logical operations on data. This block receives input from the “instr” block and gives an output to “aluin1” and “aluin2” - two inputs of the ALU.
- **aluin1, aluin2:** These represent the two inputs to the ALU. It suggests the ALU operates on two input values.
- **clk:** This represents the clock signal. It provides the timing reference for the CPU operations.
- **control:** This is the control unit. It takes input from the clk and other inputs implied in the diagram, and generates control signals. It’s responsible for coordinating the activities of other components, like the ALU, based on the instruction being executed.
- **state:** The state block receives an input from the control block. It represents the current state of the CPU, which is determined by the control unit.
- **stach:** This is a stack address register. This means that the address of memory allocated to the stack is available for use.
- **port_addr:** This represents the port address register, used for communication with external components.
- **port_write_data:** This indicates the data to be written to a port which is also used for communicating with other components.
- **memaddr:** This block represents the memory address register and is used to store the address of a memory location.
- **memval:** This represents the value to be read from memory.
- **storeval:** This represents the value to be stored in memory.
- **opcode:** This represents the operation code which specifies what action the processor needs to do.

Data Flow and Interpretation

- 1) **Instruction Fetch:** The process starts when the fetcher gets the next instruction from `ins_mem` and stores it in `instr`.

- 2) Instruction Decoding and Execution: From the instr block, the instruction is sent to several destinations.
- 1) ALU Operation: The instruction is forwarded to the alu, which receives two inputs aluin1, and aluin2 and performs the required calculation.
- 2) Control Unit: The clk signal is received by the control unit which generates state transition information.
- 3) Memory Access: The memory address (memaddr) and value to write (storeval) or to read (memval) can be derived from the instruction.
- 4) Port operations: If the instruction involves port operations, the port_addr and port_write_data will be set.
- 5) Opcode: The operation code determines what kind of calculation or other action needs to happen.

Code analysis

1) ssd_driver_tb.v

```

timescale 1ns / 1ps

module ssd_driver_tb;

    // Inputs
    reg clk;
    reg [31:0] ssd_bits;
    reg ssd_char_mode;

    // Outputs
    wire [3:0] an;
    wire [6:0] seg;
    wire dp;

    // Instantiate the ssd_driver module
    ssd_driver uut (
        .clk(clk),
        .an(an),
        .seg(seg),
        .dp(dp),
        .ssd_bits(ssd_bits),
        .ssd_char_mode(ssd_char_mode)
    );

    // Clock generation
    always begin
        clk = 0; #5; clk = 1; #5;
    end

    // Stimulus
    initial begin
        // Initialize inputs
        clk = 0;
        ssd_bits = 32'b0;
        ssd_char_mode = 0;
    end

```

File "ssd_driver_tb.v"

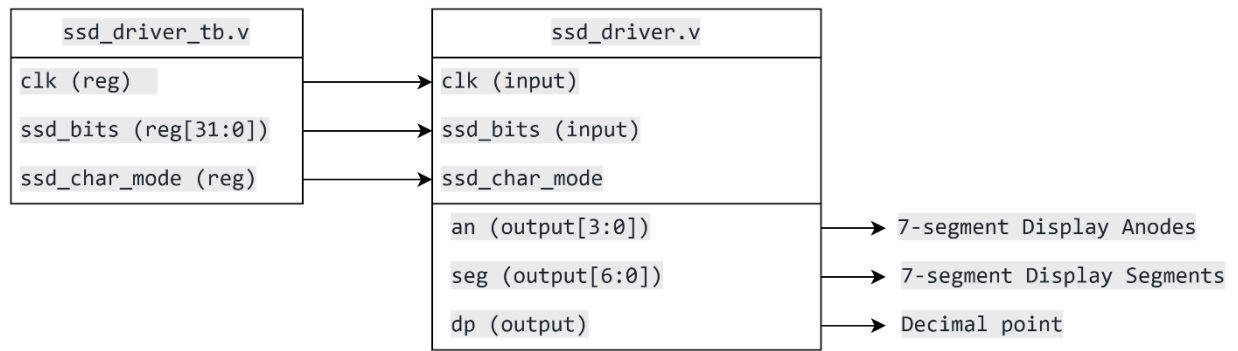
1. Function Introduction

The code is a testbench (ssd_driver_tb.v) designed to verify the functionality of an ssd_driver module. This ssd_driver module is responsible for taking a 32-bit input (ssd_bits) and driving a 7-segment display. Based on the inputs, it will control which segments to light up to display numbers or characters on the 7-segment display.

- **ssd_driver Module:**
 - **Inputs:**
 - clk: Clock signal for synchronous operation.
 - ssd_bits: A 32-bit input representing the data to be displayed on the 7-segment display(s). The data is often split into 4-bit chunks where each chunk drives an individual 7 segment display.
 - ssd_char_mode: A single bit input controlling the display mode; 0 for numeric, 1 for character mode.
 - **Outputs:**
 - an: A 4-bit output controlling which anode(s) of the 7-segment displays are activated.
 - seg: A 7-bit output controlling which segments of the activated 7-segment display are lit.
 - dp: A 1-bit output controlling the decimal point of the display.
 - **Functionality:**
 - Converts the ssd_bits input to the correct segment codes based on the display mode. In numeric mode, it displays the hex value. In character mode, it displays a simple character set.
 - Handles multiplexing of multiple 7-segment displays using the an signal.
- **ssd_driver_tb Testbench:**
 - **Function:** Provides stimulus (inputs) to the ssd_driver module, and monitors its outputs to check if the behavior is correct.
 - **Clock Generation:** Generates a clock signal with a 10ns period (5ns high, 5ns low).
 - **Test Vectors:** Provides various ssd_bits values and toggles the ssd_char_mode to verify different scenarios.
 - **Output Monitoring:** Uses the \$monitor command to display the current inputs and outputs during simulation. This helps the user to see the effect of the inputs on the outputs.

2. Design Diagram

The diagram showing the relationship between the testbench and the module:



“ssd_driver_tb.v” diagram

Internal Structure of ssd_driver:

The ssd_driver contains:

- **Clocked Logic:** Registers and logic controlled by the clk signal to manage data and multiplex the displays.
- **Multiplexer (MUX):** Selects which 7-segment display to drive based on the anode signals an.
- **Decoder:** A decoder that converts the 4-bit number from ssd_bits into the correct 7-segment codes for display. This might include logic to handle different characters in character mode.
- **Logic:** Logic for handling decimal point display and for switching between numeric and character modes.

2) tb_io_driver.v

```

timescale 1ns / 1ps

module tb_io_driver;

    // Testbench signals
    reg clk;
    reg [3:0] btn;
    reg [7:0] sw;
    wire [3:0] led;
    wire [31:0] ssd_bits;
    wire ssd_char_mode;
    reg port_read;
    reg port_write;
    reg [15:0] port_addr;
    reg [15:0] port_write_data;
    wire [15:0] port_read_data;

    // Instantiate the io_driver module
    io_driver uut (
        .clk(clk),
        .btn(btn),
        .sw(sw),
        .led(led),
        .ssd_bits(ssd_bits),
        .ssd_char_mode(ssd_char_mode),
        .port_read(port_read),
        .port_write(port_write),
        .port_addr(port_addr),
        .port_write_data(port_write_data),
        .port_read_data(port_read_data)
    );

    // Clock generation
    always begin
        #5 clk = ~clk; // 100 MHz clock period (10ns high, 10ns low)
    end
end
  
```

File “tb_io_driver.v”

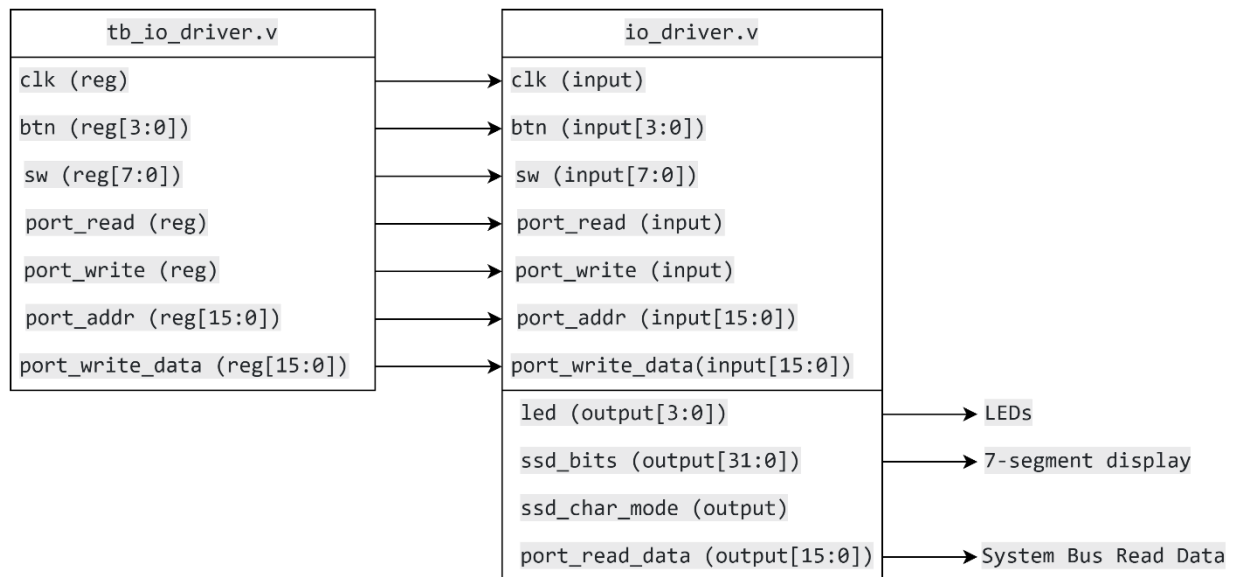
1. Function Introduction

This code represents a testbench (tb_io_driver.v) designed to verify the functionality of an io_driver module. The io_driver module acts as an interface between the system and various input/output peripherals, such as LEDs, a 7-segment display (SSD), switches, and buttons. It manages how the CPU or other system components interact with these devices using an address-based approach.

- **io_driver Module:**
 - **Inputs:**
 - clk: Clock signal for synchronous operation.
 - btn: 4-bit input representing button states.
 - sw: 8-bit input representing switch states.
 - port_read: Signal to indicate that the CPU is requesting a read from the IO system.
 - port_write: Signal to indicate that the CPU is requesting a write to the IO system.
 - port_addr: 16 bit address from the CPU indicating the IO peripheral the CPU is interacting with.
 - port_write_data: 16 bit data that the CPU wants to send to the IO peripheral.
 - **Outputs:**
 - led: 4-bit output to control LEDs.
 - ssd_bits: 32-bit output controlling the 7-segment display.
 - ssd_char_mode: Controls if the SSD shows char values or decimal/hex values.
 - port_read_data: 16 bit data to the CPU when the CPU is performing a read.
 - **Functionality:**
 - Decodes the port_addr to determine which peripheral is being accessed.
 - Handles writing data to the led and ssd_bits outputs when port_write is asserted.
 - Handles reading data from the btn and sw inputs and placing it on the port_read_data output when port_read is asserted.
- **tb_io_driver Testbench:**
 - **Function:** This testbench stimulates the io_driver module by asserting read/write signals and specific addresses. It monitors the output signals (leds, 7-segment display, and read data) and verifies whether the behavior is as expected.
 - **Clock Generation:** Generates a 100 MHz clock signal (10 ns period).

- **Stimulus:** Applies a sequence of test scenarios that write and read from the various peripherals, using addresses defined in the simulation.
- **Verification:** The test bench uses \$display to report if each test passes or fails.

2. Design Diagram



“tb_io_driver.v” diagram

Internal Structure of io_driver:

The io_driver module contains:

- **Address Decoder:** Logic that decodes the port_addr to select the specific peripheral being accessed.
- **Write Logic:** When port_write is high, the logic routes the port_write_data to the appropriate output based on port_addr. When the address is for a 7-segment display, the logic should update ssd_bits accordingly based on the port_write_data and address type (HEX, DEC, or CHAR).
- **Read Logic:** When port_read is high, the logic selects data from the switch and button inputs based on port_addr. The selected data is then driven on the port_read_data output.
- **SSD Logic:** Includes additional logic or FSM to determine what data will drive the SSD based on the input mode (char, hex, dec).

3) tb_ports.v

```

`timescale 1ns / 1ps

module tb_ports;

    // Parameters
    `include "parameters.vh"

    // Testbench signals
    reg [WORD_SIZE-1:0] portaddr, portval;
    reg clk, get_enable, set_enable;
    wire [WORD_SIZE-1:0] portout;

    // Instantiate the ports module
    ports uut (
        .portaddr(portaddr),
        .portval(portval),
        .clk(clk),
        .get_enable(get_enable),
        .set_enable(set_enable),
        .portout(portout)
    );

    // Clock generation
    always begin
        #5 clk = ~clk; // Toggle clock every 5ns
    end

    // Initial block to apply stimulus
    initial begin
        // Initialize signals
        clk = 0;
        get_enable = 0;
        set_enable = 0;
        portaddr = 0;
        portval = 0;

        // Apply test case 1: Set operation on port 1

```

File “tb_ports.v”

1. Function Introduction

This code defines a testbench module `tb_ports` that is designed to verify the functionality of a `ports` module. This `ports` module likely acts as an interface for interacting with different I/O ports or memory-mapped registers within a system. It handles setting (writing to) and getting (reading from) these ports based on address and control signals.

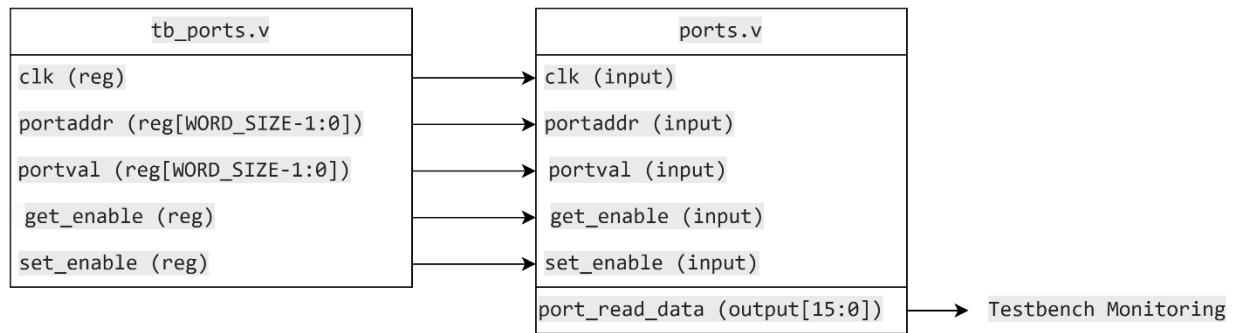
Here’s a more detailed explanation of what the `ports` module and the testbench are supposed to do:

- **ports Module:**
 - **Inputs:**
 - `portaddr`: A signal defining the address of the port to access. `WORD_SIZE` determines the width of this address, and it can be assumed the module handles as many ports as the address can allow

- portval: A signal providing data to write to a port.
- clk: Clock signal for synchronous operation.
- get_enable: A signal to initiate a read operation from a port
- set_enable: A signal to initiate a write operation to a port.
- **Outputs:**
 - portout: A signal that holds the data read from the port.
- **Functionality:**
 - Stores data written to each port at the correct address when set_enable is high.
 - Outputs the data from the specified port when get_enable is high.
 - May contain some special logic associated with some ports, especially port 0 (as suggested by the code, setting to port 0 will “halt” the machine. This is likely implemented in the module and not in the testbench.
- **tb_ports Testbench:**
 - **Function:** The testbench provides the ports module with different input stimulus to verify its operation. It generates a clock signal, sets port addresses and values, enables read and write operations, and checks the output data.
 - **Clock Generation:** Generates a clock with a 10 ns period (5 ns high, 5 ns low).
 - **Stimulus:**
 - Test Case 1: Writes a value to port 1.
 - Test Case 2: Reads from port 1.
 - Test Case 3: Writes a value to port 0 (which is assumed to be a halt).
 - **Monitoring:** Uses the \$monitor command to display the current states of inputs and outputs during the simulation, helping in debugging and observation.
 - **parameters.vh:** The testbench is using a parameter file called parameters.vh, which contains the parameter WORD_SIZE, which defines the width of the address and data buses.

2. Design Diagram

Here’s a diagram showing the interaction between the testbench and the ports module:



“tb_ports.v” diagram

Internal Structure of the ports module (Inferred):

The ports module probably contains:

- **Address Decoder:** Logic to decode the portaddr to select the appropriate port.
- **Registers/Memory:** Internal registers or memory cells to store data associated with each port.
- **Write Logic:** Logic to write portval to the selected port when set_enable is high.
- **Read Logic:** Logic to read data from the selected port and output it to portout when get_enable is high.
- **Special Logic for Port 0:** The code suggests there might be special logic associated with port 0, that when written to, will halt the machine.

Additional task (cpu_tb.v):

```

`timescale 1ns / 1ps

module cpu_tb;

    // Parameters
    parameter DATA_WIDTH = 32;
    parameter ADDR_WIDTH = 10; // Adjust as needed

    // Clock and Reset
    reg clk;
    reg reset;

    // Instruction Pointer Signals
    wire [ADDR_WIDTH-1:0] pc;
    reg branch_taken;
    reg [ADDR_WIDTH-1:0] branch_target;
    reg jump_taken;
    reg [ADDR_WIDTH-1:0] jump_target;

    // Instruction Fetch Signals
    wire [DATA_WIDTH-1:0] instr;

    // Instruction Memory (IMem) Signals
    reg [ADDR_WIDTH-1:0] imem_addr;
    wire [DATA_WIDTH-1:0] imem_data;

    // Data Memory (DMem) Signals
    reg [ADDR_WIDTH-1:0] dmem_addr;
    reg [DATA_WIDTH-1:0] dmem_data_in;
    reg dmem_we;
    wire [DATA_WIDTH-1:0] dmem_data_out;
  endmodule
  
```

File “cpu_tb.v”

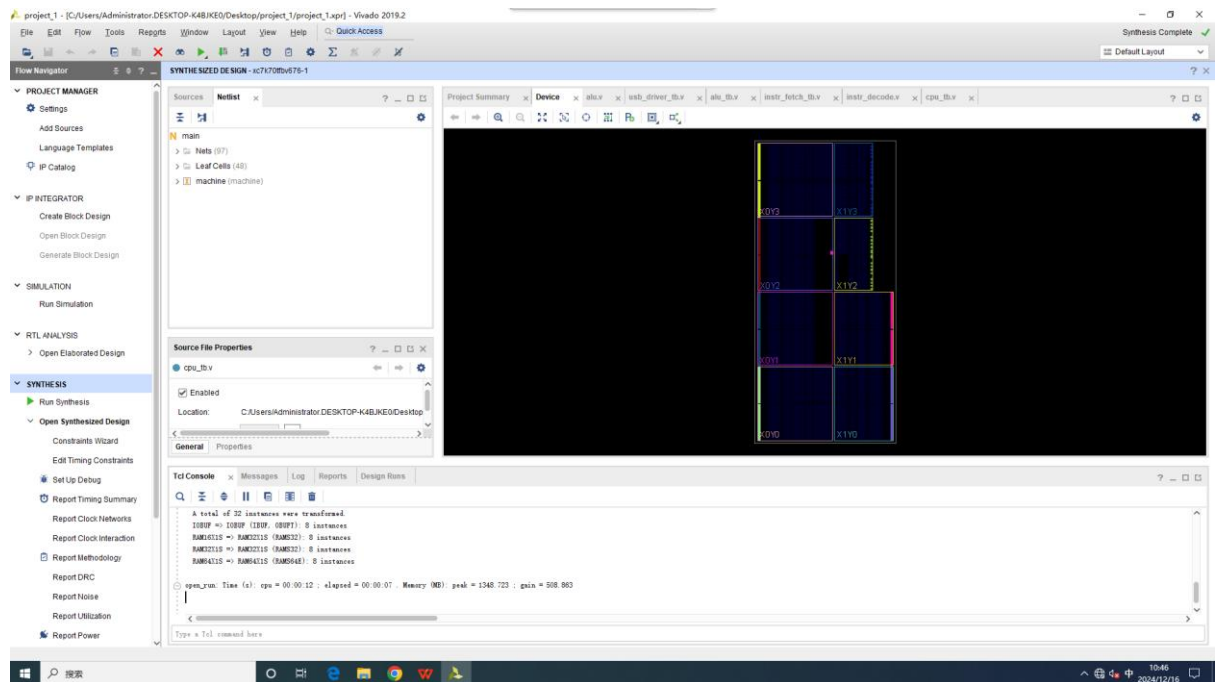
Modules to Integrate:

1. **instr_pointer.v:** Manages the program counter (PC).
2. **instr_fetch.v:** Responsible for fetching instructions from memory.
3. **instr_decode.v:** Decodes the fetched instructions.
4. **reg_stack.v:** Implements the register file.
5. **alu.v:** Arithmetic Logic Unit.
6. **control.v:** Generates control signals for the CPU based on the decoded instruction.

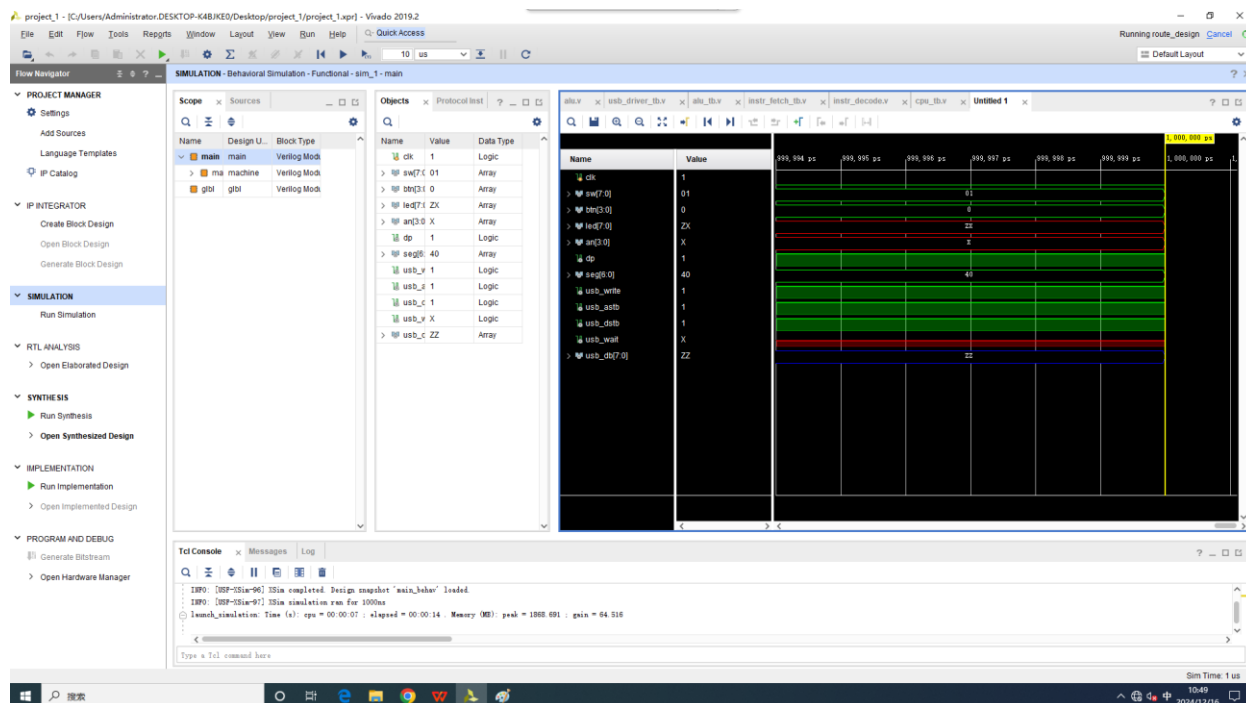
Testbench (cpu_tb.v):

- **instr_pointer.v:**
 - Inputs: clk, reset, branch_taken, branch_target, jump_taken, jump_target
 - Outputs: pc (program counter)
- **instr_fetch.v:**
 - Inputs: clk, reset, pc
 - Outputs: instr (fetched instruction)
 - (May also have an instruction memory interface as discussed previously)
- **instr_decode.v:**
 - Inputs: instr
 - Outputs: Various decoded instruction fields (e.g., opcode, rs, rt, rd, imm)
- **reg_stack.v:**
 - Inputs: clk, reset, write_enable, write_reg, write_data, read_reg_1, read_reg_2
 - Outputs: read_data_1, read_data_2
- **alu.v:**
 - Inputs: alu_op, operand_1, operand_2
 - Outputs: alu_result, zero_flag
- **control.v:**
 - Inputs: opcode
 - Outputs: Various control signals (e.g., reg_write, alu_op, mem_write, branch)

Results and diagrams



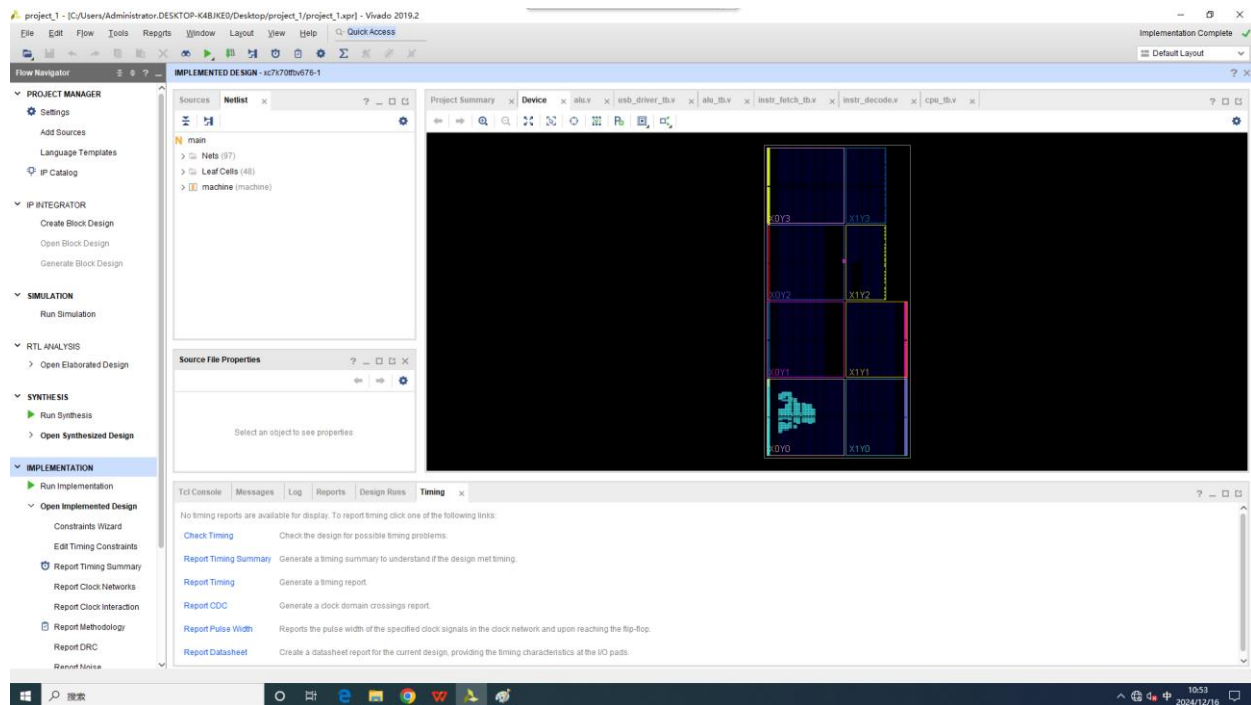
CPU design



CPU wave diagram during simulation runtime

- clk: A clock signal toggles throughout the simulation.
- sw[7:0]: The switches are initialized with a value of 0x01
- btn[3:0]: Buttons start at the state 0x0.
- led[7:0]: LEDs show high impedance. This means the leds have not yet been driven to a value.

- an[3:0]: The anode selection is an unknown value and is a high impedance output.
- dp: Decimal point is set to 1, indicating the decimal point will always be on if it is not driven to 0.
- seg[6:0]: The segments on the display are driven to the value 0x28.
- usb_write, usb_astb, usb_dstb: Control signals are active. usb_wait is in high impedance.
- usb_db[7:0]: The data on the USB is a high impedance output which is expected as it is not being driven from the system.



CPU design (implemented)

The implemented design displays the final physical arrangement with individual elements placed and the signals routed based on their logical connections. The console now focuses on timing analysis, confirming the system is operating correctly.